

10

Maestro de la Autenticación Segura

El Retorno del Token

 En proceso

Bienvenido a PassPort Inc., una startup con grandes sueños y promesas aún mayores. Después de sobrevivir a una competencia de presentaciones más estresante que ver a tu personaje favorito en una serie de televisión a punto de morir, PassPort acaba de asegurar una ronda de inversión semilla de 10 millones de dólares. ¿Las apuestas? Altísimas.

La Misión: PassPort Inc. está a punto de lanzar su aplicación insignia: una plataforma de vanguardia donde los usuarios pueden almacenar, gestionar y compartir de manera segura sus pasaportes digitales, identificaciones y otros documentos vitales. La propuesta es que PassPort será "el gestor de identidad digital más seguro y fácil de usar del planeta". Afirmación audaz, ¿verdad? Ahora, la responsabilidad de hacer realidad esa afirmación recae directamente en tus hombros.

El Escenario de Pesadilla: La aplicación se lanzará en dos semanas. Los medios están al tanto, los primeros usuarios se están registrando más rápido de lo que puedes decir "ciberseguridad", y los hackers de todo el mundo están afinando sus herramientas virtuales. Saben que si logran hackear PassPort, serán famosos, o infames. Mientras tanto, el CTO (que ha visto más startups fallidas de las que has tenido comidas calientes) te ha entregado las llaves del reino: Diseñar e implementar el Sistema de Gestión de Sesiones y Autenticación.

Sin presión, ¿verdad?

Conceptos que debes (no, que tenés que) aprender:

Antes de sumergirte, aquí tienes lo que necesitas saber:

1. **Sesiones vs. Tokens:** No, no son lo mismo, y sí, necesitas conocer la diferencia.
2. **Gestión de Cookies:** Porque, ¿qué es una sesión sin cookies? Triste y solitaria.
3. **JWT (JSON Web Tokens):** La brillante y casi mágica solución para la autenticación sin estado. Spoiler: no todo es arcoíris y unicornios.
4. **Cifrado y Hashing:** Asegurate de que tus contraseñas no se almacenen en texto plano. En serio, no seas ese desarrollador.
5. **Cross-Site Scripting (XSS) y Cross-Site Request Forgery (CSRF):** Los peores enemigos de tu futura aplicación.
6. **Control de Acceso Basado en Roles (RBAC):** Porque no quieres que todos, incluida la abuela, tengan derechos de administrador.
7. **Cabeceras HTTP para Seguridad:** Familiarízate con Authorization, Set-Cookie y sus amigos.

¡¡ Empecemos !!

Inicio de sesión con nombre de usuario y contraseña

Para empezar, necesitas construir un flujo de inicio de sesión que sea intuitivo y seguro. Esto significa que los usuarios deben poder registrarse fácilmente, y su información debe estar protegida desde el primer momento. El proceso debe ser claro: los usuarios ingresan su correo electrónico y crean una contraseña. El registro e inicio de sesión deben ser simples y directos. Nadie quiere pasar por un laberinto solo para acceder a su cuenta.

El uso de hashing

Almacenar contraseñas en texto plano es como dejar la llave de la puerta bajo la alfombra: cualquiera podría encontrarla. En su lugar, **debes usar hashing**, un proceso que convierte la contraseña original en un código único e irreversible. Esto asegura que, incluso si alguien accediera a la base de datos, no podría ver las contraseñas reales. En otras palabras, estás guardando la llave en una caja fuerte, y solo el dueño legítimo tiene la combinación secreta para abrirla.

Gestión de Sesiones

Ofrece a los usuarios la opción de elegir entre sesiones persistentes con cookies o sesiones sin estado con JWT al iniciar sesión.

Sesiones Persistentes (Usando Cookies)

Las sesiones persistentes permiten a los usuarios permanecer conectados incluso después de cerrar el navegador. Esto se logra mediante cookies que almacenan un identificador de sesión en el navegador del usuario.

- Iniciar sesión: Cuando el usuario inicia sesión, el servidor crea una sesión y envía una cookie con un identificador de sesión.
- Mantener Conexión: En cada solicitud futura, el navegador envía esta cookie al servidor, que la utiliza para identificar al usuario y mantener la sesión activa.
- Cerrar Sesión: Al cerrar sesión, el servidor elimina la sesión y borra la cookie.

Sesiones sin Estado (Usando JWT)

Las sesiones sin estado usan JWT (JSON Web Tokens) para autenticar usuarios sin mantener información de sesión en el servidor. El token incluye toda la información necesaria sobre el usuario.

- Iniciar sesión: Al iniciar sesión, el servidor genera un JWT con la información del usuario y lo envía al cliente.
- Mantener Conexión: El cliente incluye el JWT en el encabezado Authorization de cada solicitud. El servidor verifica el token para autenticar al usuario.
- Cerrar Sesión: El cliente simplemente elimina el JWT; el servidor no mantiene estado de la sesión.

Control de Acceso Basado en Roles (RBAC)

El control de acceso basado en roles (RBAC) permite gestionar permisos según el rol del usuario. En este caso, habrá dos roles principales: Usuario y Administrador.

Primero define los roles.

- **Usuario:** Acceso básico a sus propios datos y funcionalidades estándar.
- **Administrador:** Acceso a funciones sensibles como eliminar datos de usuario y ver intentos fallidos de inicio de sesión.

Luego asigna los roles, al crear o editar usuarios, asigna un rol específico (Usuario o Administrador)

- **Para usuarios estándar:** Limita el acceso a ciertas rutas y funcionalidades usando middleware o validaciones basadas en el rol del usuario.
- **Para administradores:** Permite acceso a funciones adicionales como eliminar datos de otros usuarios y revisar registros de seguridad.

Protección contra Vulnerabilidades Comunes

- **Implementa el uso de Cifrado y Hashing:** No olvides usar algoritmos de hashing (como bcrypt) para almacenar contraseñas de forma segura. Y cifra los datos sensibles en los tokens para proteger la información del usuario.
- **Cross-Site Scripting (XSS):** Filtra y escapa las entradas del usuario para prevenir que scripts maliciosos se ejecuten en el navegador.
- **Tokens CSRF (Cross-Site Request Forgery):** Usa tokens únicos para validar las solicitudes que cambian el estado. Esto evita que atacantes envíen solicitudes en nombre del usuario sin su conocimiento.
- **Prevención de Ataques de Fuerza Bruta:** Implementa mecanismos para limitar el número de intentos de inicio de sesión, como bloqueos temporales después de varios intentos fallidos.
- **Cookies Seguras:** Configura las flags HTTP-only y Secure en las cookies para prevenir el acceso del lado del cliente y la transmisión a través de conexiones no HTTPS.

Nota Final: Este desafío no es solo sobre escribir código. Se trata de pensar como un hacker, un sysadmin, un gerente de producto, y un usuario, todo al mismo tiempo. Si te sientes abrumado, es normal. Si no te sientes abrumado, estás haciendo algo mal.

El reloj está corriendo. Los inversores están observando. El mundo está esperando para ver si PassPort será el próximo unicornio o el próximo chiste. Remangate, sumergite en la documentación, escribi esas pruebas, y que el código esté siempre a tu favor.

Resumen de Requerimientos

Requerimientos Obligatorios:

 *Los requerimientos obligatorios deben ser completados en su totalidad o el ejercicio no se considera válido.*

1. Implementa un flujo de registro y autenticación que permita a los usuarios crear una cuenta con correo electrónico y contraseña.
2. Almacena las contraseñas de forma segura usando un algoritmo de hashing (por ejemplo, bcrypt) para convertir las contraseñas en un código único e irreversible.
3. Implementa la creación, mantenimiento y eliminación de sesiones con cookies. La cookie debe almacenar un identificador de sesión.
4. Implementa la autenticación basada en JWT, generando y validando tokens que contengan la información del usuario.
5. Define al menos dos roles: Usuario y Administrador. Implementa la lógica para restringir el acceso a ciertas rutas y funcionalidades basadas en el rol del usuario.
6. Usa algoritmos de cifrado para proteger datos sensibles en los tokens y hash para contraseñas.
7. Filtra y escapa las entradas del usuario para prevenir la ejecución de scripts maliciosos.
8. Usa tokens CSRF para validar las solicitudes que cambian el estado.
9. Implementa limitación de intentos de inicio de sesión, como bloqueos temporales después de múltiples intentos fallidos.
10. Configura las flags HTTP-only y Secure en las cookies.

Requerimientos Opcionales:

 *Los requerimientos opcionales quedan a criterio del participante, su total y correcta implementación pueden influir en obtener una evaluación excepcional.*

1. Implementa una interfaz de usuario con una experiencia de usuario fluida para el registro e inicio de sesión, usando HTML y CSS.
2. Escribe pruebas automatizadas para verificar el correcto funcionamiento del sistema de autenticación y la seguridad (opcional pero altamente recomendado).
3. Agrega un flujo seguro para la recuperación de contraseñas. Los usuarios deberían poder solicitar un enlace de restablecimiento de contraseña enviado por correo electrónico.

Consideraciones para el ejercicio

 *El objetivo principal de este desafío es que aprendas a crear un sistema de autenticación seguro y eficiente, enfocándote en proteger los datos de los usuarios y garantizar un acceso controlado. Te ayudará a desarrollar habilidades clave para implementar medidas de seguridad robustas en aplicaciones web.*

1. Antes de comenzar a codificar, asegurate de entender completamente los requisitos tanto obligatorios como opcionales. Documenta el flujo de autenticación y los roles que implementarás.
2. Considera cómo escalarás tu sistema de autenticación a medida que crezca el número de usuarios. La arquitectura debe permitir añadir más características sin grandes modificaciones.
3. Asegurate de que el flujo de registro e inicio de sesión sea intuitivo y fácil de usar. Una buena experiencia de usuario es crucial para la adopción de la aplicación.
4. Realiza pruebas de seguridad para identificar y corregir vulnerabilidades. Considera realizar pruebas de penetración para evaluar la resistencia de tu sistema.
5. Implementa la solución y luego optimízala:
 - Define los requisitos y objetivos del sistema de autenticación.
 - Diseña la arquitectura de autenticación y autorización.
 - Implementa el flujo de registro de usuarios.
 - Configura el inicio de sesión con la validación de credenciales.
 - Implementa hashing de contraseñas
 - Asegurate de que las contraseñas se almacenen de manera segura.
 - Implementa sesiones persistentes con cookies (incluye configuración de seguridad en cookies).
 - Implementa sesiones sin estado con JWT (incluye generación y verificación de tokens).
 - Define roles (Usuario y Administrador).
 - Implementa permisos y restricciones basadas en roles.
 - Implementa protección contra XSS y CSRF
 - Implementa mecanismos de prevención de ataques de fuerza bruta.
 - Asegurate que todo funcione correctamente
 - Optimiza el programa